

Московский авиационный институт
(национальный исследовательский университет)

Лабораторная работа №2 по курсу «Информационный поиск»

Студент: Пирязев М.А.
Преподаватель: А.А. Кухтичев
Группа М8О-407Б-22
Дата: 14.11.2025
Оценка:
Подпись:

Москва, 2025

Задание

Необходимо реализовать поискового робота (компоненты обкатки документов) со следующими требованиями:

- робот принимает единственный аргумент: путь до YAML-конфига;
- YAML-конфиг содержит параметры подключения к базе данных в секции db и параметры логики робота в секции logic (в том числе задержку между запросами);
- робот сохраняет в MongoDB документы с полями: нормализованный url, «сырой» текст документа (raw_html), название источника (source), дата обкатки last_crawled_at в формате Unix timestamp;
- робот может быть остановлен в любой момент и при повторном запуске продолжает работу с места остановки;
- робот должен уметь переобкатывать документы, которые уже есть в базе, обновляя их только в случае изменения содержимого.

Описание

В рамках лабораторной работы реализован робот на языке Python, который поддерживает два основных сценария работы: первичную загрузку документов (seed) и переобкатку (recrawl). В режиме seed робот заполняет MongoDB документами, используя локальные дампы выбранных источников (Wikipedia и Stack Overflow), выполняя потоковый разбор данных для снижения потребления памяти. В режиме recrawl робот выбирает документы, обкатанные более N дней назад, скачивает актуальную версию по URL и обновляет документ только в случае изменения содержимого (проверка выполняется сравнением хэшей содержимого).

Для обеспечения повторяемости и удобства эксплуатации поведение робота задаётся через YAML-конфиг: параметры базы данных, таймауты, количество повторных попыток, задержки между запросами, лимиты и настройки чекпоинтов.

Журнал выполнения и решение проблем

В ходе выполнения лабораторной работы был реализован и протестирован поисковый робот. Сначала был подготовлен YAML-конфиг, в котором заданы параметры подключения к MongoDB, основные параметры логики работы (задержки, таймауты, повторы) и описание источников данных. Далее была реализована первичная загрузка корпуса (seed) из дампов Wikipedia и Stack Overflow с сохранением документов в MongoDB в требуемом формате: нормализованный URL, «сырой» текст документа, название источника и время обкачки в Unix timestamp. Для обеспечения устойчивости работы была добавлена система чекпоинтов (resume) — состояние обработки периодически сохраняется в файл `corpus/.crawler_state.json`, благодаря чему работа можно остановить в любой момент и затем продолжить выполнение без потери прогресса. Также реализован режим переобкачки (recrawl): робот выбирает устаревшие документы по полю `last_crawled_at`, скачивает актуальную версию по URL и обновляет запись только в случае изменения содержимого.

В процессе реализации возникло несколько практических проблем. Во-первых, исходные дампы имеют большой объём, поэтому был выбран потоковый разбор XML через `iterparse`, что позволяет обрабатывать данные без загрузки всего файла в память. Во-вторых, документы из источников содержат разметку (wikitext у Wikipedia и HTML у Stack Overflow); это не является ошибкой, так как по требованиям лабораторной работы необходимо сохранять именно «сырой» контент в поле `raw_html`, а этап очистки текста логичнее выполнять позже при индексации. В-третьих, для проверки механизма переобкачки потребовалось создать ситуацию с «устаревшими» документами: для этого вручную изменялись значения `last_crawled_at` у части записей, после чего робот корректно находил такие документы и выполнял проверку обновлений. Наконец, при подготовке тестового сценария обновления части документов было учтено, что `updateMany()` в MongoDB не поддерживает ограничение `limit`; вместо этого использовался подход с выборкой ограниченного набора `_id` и последующим обновлением.

Исходный код

Основная часть программы реализована в скрипте `crawler.py`, который запускается из командной строки и принимает единственный параметр — путь к YAML-конфигу. После старта программа загружает конфигурацию, настраивает логирование и подключается к MongoDB, создавая необходимые индексы (в первую очередь уникальный индекс по `url`). Далее робот определяет режим работы: если база пуста, выполняется первичная загрузка корпуса (`seed`), а если в базе есть документы старше заданного порога, запускается переобкатка (`recrawl`). Такой подход позволяет использовать один и тот же скрипт как для первоначального наполнения базы, так и для поддержания её актуальности.

В режиме `seed` реализованы два независимых обработчика источников — для Wikipedia и для Stack Overflow. Для Wikipedia используется потоковый разбор большого XML-дампа в архиве `.bz2`, при котором извлекаются страницы, отбрасываются редиректы и служебные пространства имён, а затем формируется документ с обязательными полями `url`, `raw_html`, `source`, `last_crawled_at`. Для Stack Overflow аналогично выполняется потоковый разбор `Posts.xml`, при этом выбираются только вопросы (`PostTypeId=1`), поскольку они являются наиболее информативной частью корпуса. Полученные документы вставляются в MongoDB; дубликаты по `url` не приводят к остановке программы благодаря уникальному индексу и обработке соответствующих исключений.

Для обеспечения устойчивости работы реализована система чекпоинтов: периодически сохраняется состояние обработки в файл `corpus/.crawler_state.json`. Это позволяет безопасно прерывать выполнение и при повторном запуске продолжать загрузку с места остановки, не обрабатывая уже пройденные фрагменты дампа повторно. В режиме `recrawl` робот выбирает документы, у которых значение `last_crawled_at` слишком старое, скачивает актуальную версию по `url` и сравнивает новое содержимое со старым (по хэшу). Обновление в базе выполняется только при обнаружении изменений, после чего фиксируется новое время обкатки. Таким образом программа удовлетворяет требованиям по сохранению «сырых» документов, поддержке `resume` и переобкатке с учётом изменений.

Выводы

Выполнив лабораторную работу по курсу «Информационный поиск», я на практике понял, как реализуется поисковый робот для формирования и поддержания корпуса документов. В ходе работы я научился строить процесс обкатки так, чтобы он был воспроизводимым и управляемым через YAML-конфиг, а также устойчивым к остановкам за счёт чекпоинтов и возобновления выполнения. Отдельно стало ясно, почему на этом этапе важно сохранять «сырой» контент документа и не пытаться полностью очищать текст сразу, поскольку подготовка текста для поиска относится к этапу индексации. Наконец, я реализовал переобкатку с обновлением только изменившихся документов и увидел, как это снижает лишнюю работу и делает систему ближе к реальным поисковым решениям.